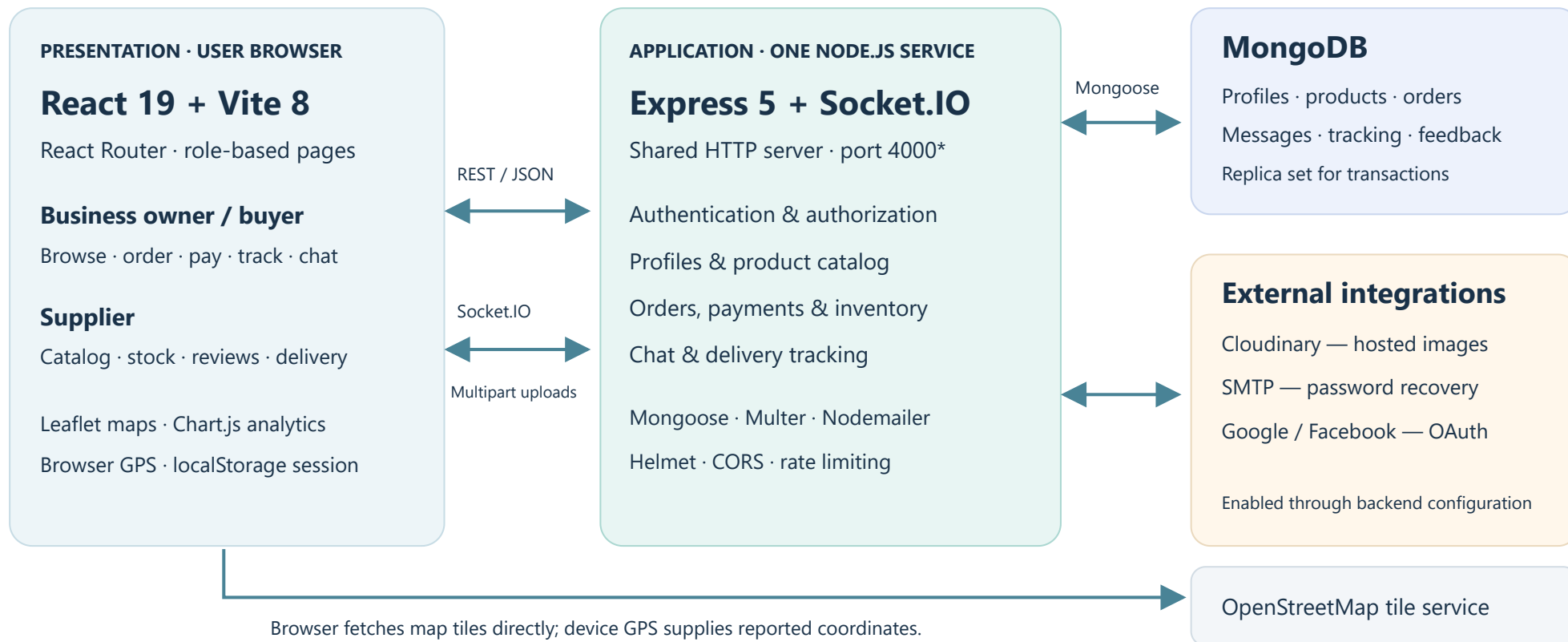


System architecture

A supplier-to-business marketplace connecting product discovery, ordering, manual payment review, messaging, and delivery tracking through one web application.



Logical architecture verified against the workspace on 7 October 2026. *4000 is the default API port. Arrows show communication, not separate deployed microservices.

Three layers, with real-time updates

Component	Responsibility	Implementation reference
Browser application	Public pages, registration, sign-in, role-gated dashboards, supplier discovery, product browsing and order screens.	frontend/src/App.jsx frontend/src/pages/
Client API & session	Fetch requests with bearer tokens; multipart product and payment uploads. Token and user data are stored in localStorage.	frontend/src/api/client.js frontend/src/lib/session.js
REST application	Identity, profiles, products, conversations, messages, orders, payment review, tracking, sales reads and feedback.	backend/server.js
Order rules	Validate payment inputs and state transitions; update stock and order status in one MongoDB transaction.	backend/payment.js backend/order-status.js
Real-time channel	Authenticated user and conversation rooms deliver messages, typing, read receipts, presence, order changes and location updates.	Socket.IO in backend/server.js Socket.IO client in frontend components
Persistence & media	Mongoose models store application records. Cloudinary stores product, profile and QR images; MongoDB stores private payment proof buffers.	Model and upload definitions in backend/server.js

Identity and access

Passwords use salted script hashes. The backend issues a custom HMAC-SHA256 signed bearer token with a seven-day expiry. Google and Facebook sign-in are available when configured.

Server checks enforce account roles, ownership and conversation membership. Socket connections also validate their token.

Request protection

Helmet security headers, an origin allowlist, endpoint rate limits and input validation protect the API boundary. JSON requests are limited to 100 KB.

Image uploads accept JPEG, PNG and WebP, up to 5 MB per file. Payment proof access is limited to the order's buyer and supplier.

MongoDB is the application system of record

Orders are stored using the **Sale** model. References connect buyers, suppliers, products and their conversations; payment details are embedded in the order.

Model	Key stored data	Relationships
Profile	Business/supplier role, identity, contact details, password hash, social IDs and profile photo URL.	One supplier → many products. One profile → many purchases or supplied orders.
Product	Name, category, price, stock, hosted image URLs and GCash QR URL.	supplierId → Profile
Sale (order)	Quantity, total, delivery details, status, payment method and payment review data, including protected receipt buffer.	buyerId / supplierId → Profile productId → Product
TrackingPoint	Reported delivery position, accuracy and timestamp.	orderId → Sale (logical association) One order → many recorded positions.
Conversation	Participant IDs and last-message time.	participantIds → Profile[]
Message	Text, sender, recipient, read time and timestamps.	conversationId → Conversation senderId / recipientId → Profile
Feedback	Sender name, email, message and developer email.	Independent feedback record.

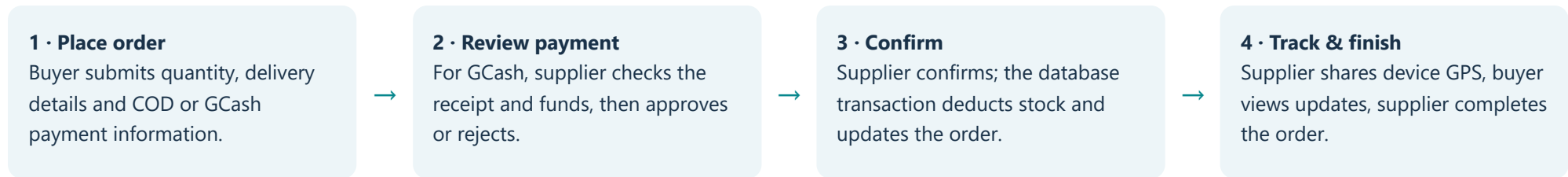
Inventory consistency

Pending orders do not reserve stock. Supplier confirmation decrements stock and changes order status atomically. Cancelling a confirmed order restores stock. Repeating the same status change does not deduct stock again.

Storage boundaries

Public-facing image URLs refer to Cloudinary. Payment receipts remain in MongoDB, excluded from normal order JSON and normal projections. Tracking positions are stored separately from order documents.

From checkout to delivery



Communication and tracking

- **Chat:** messages are submitted through REST, persisted in MongoDB, then emitted to the authorized conversation room.
- **Location:** supplier browser GPS is uploaded through the order location endpoint, stored as TrackingPoint records, and broadcast to buyer/supplier rooms.
- **Refresh:** GPS uploads are throttled to once per ten seconds per open order. Tracking also refreshes every thirty seconds and on reconnect.
- **Tracking ID:** CD- followed by the order ID. Lookup requires an authenticated order participant.

Payment behavior

GCash uses manual QR payment and supplier verification. Approval is required before a GCash order can be confirmed or completed. A rejected receipt can be corrected and resubmitted. Payment review and order confirmation are separate actions.

Local development

Vite serves the browser application on port 5173 and proxies /api to the Node API on port 4000. The API runs separately and connects to MongoDB.

Production deployment requirements

- Serve the Vite build from a frontend host with SPA route fallback; run the Node HTTP/Socket.IO service on a backend host.
- Set VITE_API_URL to the API origin and CLIENT_ORIGIN to the allowed frontend origin. Enable HTTPS and Socket.IO connectivity.
- Use MongoDB Atlas or a replica set for order transactions. Keep database, auth, OAuth, SMTP and Cloudinary secrets in the backend environment.

This is the deployment shape supported by the code. An actual production host or running deployment was not verified.

Operational boundaries: GCash is not connected to a payment gateway. Delivery tracking requires the supplier's page to remain open and the device awake; it is foreground browser GPS. Presence and default rate-limit state are in-process; a multi-instance deployment would need shared coordination.

Source basis: backend/server.js; backend/config/env.js; backend/payment.js; backend/order-status.js; frontend/src/App.jsx; frontend/src/api/client.js; frontend/src/components/OrderTracking.jsx; frontend/vite.config.js; frontend/README.md; PAYMENTS.md; TRACKING.md. Prepared from source inspection, not a live integration audit.